

# retail-demand-forecasting

September 27, 2026

## 1 Retail Demand Forecasting — Store Sales (Corporacion Favorita)

Data Science Internship Project — Final Presentation Notebook — Codec Technologies

Forecasting daily unit sales per store x product-family for a 54-store Ecuadorian grocery chain, using historical sales, promotions, holidays, and oil price as inputs. Comparing naive baselines, ARIMA, Prophet, Random Forest, XGBoost, and an LSTM, evaluated on RMSE, MAPE, WAPE, and forecast bias.

Dataset: [Store Sales - Time Series Forecasting \(Kaggle competition\)](#)

Structured as Ask -> Prepare -> Process -> Analyze -> Share -> Act.

---

**Contents** 1. Ask — Define the Business Task 2. Prepare — Source and Assess the Data (*expanded dataset overview + EDA*) 3. Process — Clean and Engineer Features 4. Analyze — Model Building and Comparison (experiment matrix + tuning + deployable final-model selection) 5. Share — Visualize and Communicate Findings 6. Act — Recommendations and Next Steps

### 1.1 1. Ask - Define the Business Task

Goal: forecast daily unit sales for each store x product-family combination over the next 15 days, to support inventory replenishment decisions.

The forecaster isn't the decision-maker here - the point is an unbiased, useful forecast planners can act on, not a number tuned to look good on one metric.

Target: company-wide WAPE  $\leq 15\%$  and RMSE below the naive-baseline RMSE on the held-out 15-day window, for at least 80% of product families.

```
[ ]: import os
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

```

import plotly.express as px

from statsmodels.tsa.statespace.sarimax import SARIMAX

from prophet import Prophet

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import TimeSeriesSplit, RandomizedSearchCV

import xgboost as xgb

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

ASSETS_DIR = "assets"
os.makedirs(ASSETS_DIR, exist_ok=True)

print("Environment ready.")

```

Environment ready.

## 1.2 2. Prepare - Source and Assess the Data

Downloaded via the Kaggle competition CLI into `data/` - 6 linked files. Comprehensive (~3M rows) and Reliable, but `oil.csv` has gaps that need addressing, and this records realized *sales*, not true *demand* - stockouts can under-report demand on some days. Worth flagging as a limitation up front rather than assuming `sales = demand`.

```

[ ]: DATA_DIR = "data"

train = pd.read_csv(
    f"{DATA_DIR}/train.csv",
    parse_dates=["date"],
    dtype={"store_nbr": "int16", "family": "category", "onpromotion": "int32",
    ↪ "sales": "float32"},
)
test = pd.read_csv(
    f"{DATA_DIR}/test.csv",
    parse_dates=["date"],
    dtype={"store_nbr": "int16", "family": "category", "onpromotion": "int32"},
)

```

```

)
stores = pd.read_csv(
    f"{DATA_DIR}/stores.csv",
    dtype={"store_nbr": "int16", "city": "category", "state": "category",
    ↪ "type": "category", "cluster": "int16"},
)
oil = pd.read_csv(f"{DATA_DIR}/oil.csv", parse_dates=["date"],
    ↪ dtype={"dcoilwtico": "float32"})
holidays = pd.read_csv(
    f"{DATA_DIR}/holidays_events.csv",
    parse_dates=["date"],
    dtype={"type": "category", "locale": "category", "locale_name": "category"},
)
transactions = pd.read_csv(
    f"{DATA_DIR}/transactions.csv",
    parse_dates=["date"],
    dtype={"store_nbr": "int16", "transactions": "int32"},
)

print(f"train:      {train.shape}")
print(f"test:       {test.shape}")
print(f"stores:      {stores.shape}")
print(f"oil:         {oil.shape}")
print(f"holidays:    {holidays.shape}")
print(f"transactions: {transactions.shape}")

```

```

train:      (3000888, 6)
test:       (28512, 5)
stores:     (54, 5)
oil:        (1218, 2)
holidays:  (350, 6)
transactions: (83488, 3)

```

### 1.2.1 Dataset overview - one function, applied to each file

```

[ ]: def dataset_overview(data, name):
    """Shape, dtypes, memory, missing values, duplicates, date range - the
    same one-stop summary used for the fraud-detection project, generalized
    to any of the 6 files here."""
    print("=" * 60)
    print(f"{name}: {data.shape[0]:,} rows x {data.shape[1]} columns")
    print(f"Memory usage: {data.memory_usage(deep=True).sum() / 1e6:.1f} MB")

    missing = data.isnull().sum()
    if missing.sum() > 0:
        print("\nMissing values:")
        print(missing[missing > 0])

```

```

else:
    print("\nMissing values: none")

    print(f"Duplicate rows: {data.duplicated().sum():,}")

    if "date" in data.columns:
        print(f>Date range: {data['date'].min().date()} to {data['date'].max().
↪date()}")

for name, data in [
    ("train", train), ("test", test), ("stores", stores),
    ("oil", oil), ("holidays", holidays), ("transactions", transactions),
]:
    dataset_overview(data, name)
    print()

```

```

=====
train: 3,000,888 rows x 6 columns
Memory usage: 81.0 MB

```

```

Missing values: none
Duplicate rows: 0
Date range: 2013-01-01 to 2017-08-15

```

```

=====
test: 28,512 rows x 5 columns
Memory usage: 0.7 MB

```

```

Missing values: none
Duplicate rows: 0
Date range: 2017-08-16 to 2017-08-31

```

```

=====
stores: 54 rows x 5 columns
Memory usage: 0.0 MB

```

```

Missing values: none
Duplicate rows: 0

```

```

=====
oil: 1,218 rows x 2 columns
Memory usage: 0.0 MB

```

```

Missing values:
dcoilwtico    43
dtype: int64
Duplicate rows: 0

```

Date range: 2013-01-01 to 2017-08-31

=====

holidays: 350 rows x 6 columns

Memory usage: 0.0 MB

Missing values: none

Duplicate rows: 0

Date range: 2012-03-02 to 2017-12-26

=====

transactions: 83,488 rows x 3 columns

Memory usage: 1.2 MB

Missing values: none

Duplicate rows: 0

Date range: 2013-01-01 to 2017-08-15

## 1.2.2 Store network summary

```
[ ]: def store_network_summary(stores_df):  
    """Counts by city, state, type, and cluster - the dimensions any sales  
    breakdown below will be sliced along."""  
    print(f"Total stores: {len(stores_df)}")  
  
    print(f"\nStores per type:")  
    print(stores_df["type"].value_counts())  
  
    print(f"\nStores per cluster (top 5 of {stores_df['cluster'].nunique()}  
↪clusters):")  
    print(stores_df["cluster"].value_counts().head())  
  
    print(f"\nStores per state (top 10 of {stores_df['state'].nunique()}  
↪states):")  
    print(stores_df["state"].value_counts().head(10))  
  
    print(f"\nStores per city (top 10 of {stores_df['city'].nunique()} cities):  
↪")  
    print(stores_df["city"].value_counts().head(10))  
  
store_network_summary(stores)
```

Total stores: 54

Stores per type:

type

```
D      18
C      15
A       9
B       8
E       4
Name: count, dtype: int64
```

```
Stores per cluster (top 5 of 17 clusters):
cluster
3       7
6       6
10      6
15      5
13      4
Name: count, dtype: int64
```

```
Stores per state (top 10 of 16 states):
state
Pichincha                19
Guayas                   11
Azuay                     3
Manabi                    3
Santo Domingo de los Tsachilas  3
Cotopaxi                  2
El Oro                    2
Los Rios                  2
Tungurahua                2
Bolivar                   1
Name: count, dtype: int64
```

```
Stores per city (top 10 of 22 cities):
city
Quito                18
Guayaquil            8
Cuenca                3
Santo Domingo        3
Ambato                2
Latacunga             2
Machala              2
Manta                 2
Babahoyo              1
Cayambe               1
Name: count, dtype: int64
```

### 1.2.3 Numeric feature summary statistics

```
[5]: def numeric_summary(data, cols):  
    """describe() plus skewness/kurtosis - useful for spotting the heavy  
    right-skew typical of retail sales data."""  
    summary = data[cols].describe().T  
    summary["skew"] = data[cols].skew()  
    summary["kurtosis"] = data[cols].kurt()  
    return summary.round(2)  
  
print(numeric_summary(train, ["sales", "onpromotion"]))  
print()  
print(numeric_summary(oil, ["dcoilwtico"]))  
print()  
print(numeric_summary(transactions, ["transactions"]))
```

|             | count     | mean   | std     | min | 25% | 50%  | 75%    | max      | \ |
|-------------|-----------|--------|---------|-----|-----|------|--------|----------|---|
| sales       | 3000888.0 | 357.78 | 1102.00 | 0.0 | 0.0 | 11.0 | 195.85 | 124717.0 |   |
| onpromotion | 3000888.0 | 2.60   | 12.22   | 0.0 | 0.0 | 0.0  | 0.00   | 741.0    |   |

|             | skew  | kurtosis |
|-------------|-------|----------|
| sales       | 7.36  | 154.56   |
| onpromotion | 11.17 | 240.87   |

|            | count  | mean  | std   | min   | 25%  | 50%   | 75%   | max    | skew | \ |
|------------|--------|-------|-------|-------|------|-------|-------|--------|------|---|
| dcoilwtico | 1175.0 | 67.71 | 25.63 | 26.19 | 46.4 | 53.19 | 95.66 | 110.62 | 0.32 |   |

|            | kurtosis |
|------------|----------|
| dcoilwtico | -1.61    |

|              | count   | mean   | std    | min | 25%    | 50%    | 75%    | max    | \ |
|--------------|---------|--------|--------|-----|--------|--------|--------|--------|---|
| transactions | 83488.0 | 1694.6 | 963.29 | 5.0 | 1046.0 | 1393.0 | 2079.0 | 8359.0 |   |

|              | skew | kurtosis |
|--------------|------|----------|
| transactions | 1.52 | 2.57     |

### 1.2.4 Categorical feature summary

```
[6]: def categorical_summary(data, col):  
    """Value counts + percentage share for a categorical column."""  
    counts = data[col].value_counts()  
    pct = data[col].value_counts(normalize=True) * 100  
    return pd.DataFrame({"count": counts, "pct_%": pct.round(2)})  
  
categorical_summary(train, "family")
```

```
[6]:
```

|                            | count | pct_% |
|----------------------------|-------|-------|
| family                     |       |       |
| AUTOMOTIVE                 | 90936 | 3.03  |
| BABY CARE                  | 90936 | 3.03  |
| BEAUTY                     | 90936 | 3.03  |
| BEVERAGES                  | 90936 | 3.03  |
| BOOKS                      | 90936 | 3.03  |
| BREAD/BAKERY               | 90936 | 3.03  |
| CELEBRATION                | 90936 | 3.03  |
| CLEANING                   | 90936 | 3.03  |
| DAIRY                      | 90936 | 3.03  |
| DELI                       | 90936 | 3.03  |
| EGGS                       | 90936 | 3.03  |
| FROZEN FOODS               | 90936 | 3.03  |
| GROCERY I                  | 90936 | 3.03  |
| GROCERY II                 | 90936 | 3.03  |
| HARDWARE                   | 90936 | 3.03  |
| HOME AND KITCHEN I         | 90936 | 3.03  |
| HOME AND KITCHEN II        | 90936 | 3.03  |
| HOME APPLIANCES            | 90936 | 3.03  |
| HOME CARE                  | 90936 | 3.03  |
| LADIESWEAR                 | 90936 | 3.03  |
| LAWN AND GARDEN            | 90936 | 3.03  |
| LINGERIE                   | 90936 | 3.03  |
| LIQUOR,WINE,BEER           | 90936 | 3.03  |
| MAGAZINES                  | 90936 | 3.03  |
| MEATS                      | 90936 | 3.03  |
| PERSONAL CARE              | 90936 | 3.03  |
| PET SUPPLIES               | 90936 | 3.03  |
| PLAYERS AND ELECTRONICS    | 90936 | 3.03  |
| POULTRY                    | 90936 | 3.03  |
| PREPARED FOODS             | 90936 | 3.03  |
| PRODUCE                    | 90936 | 3.03  |
| SCHOOL AND OFFICE SUPPLIES | 90936 | 3.03  |
| SEAFOOD                    | 90936 | 3.03  |

### 1.2.5 How many series, and are they complete?

```
[ ]: n_stores = train["store_nbr"].nunique()
      n_families = train["family"].nunique()
      n_series = train.groupby(["store_nbr", "family"], observed=True).ngroups

      print(f"Stores: {n_stores}")
      print(f"Product families: {n_families}")
      print(f"Store x family series: {n_series}")
```

```

full_date_range = pd.date_range(train["date"].min(), train["date"].max(),
    ↪freq="D")
expected_days = len(full_date_range)

series_lengths = train.groupby(["store_nbr", "family"], observed=True)["date"].
    ↪nunique()
incomplete_series = (series_lengths != expected_days).sum()

print(f"\nExpected days per complete series: {expected_days}")
print(f"Series with at least one missing day: {incomplete_series} of {n_series}
    ↪"
        f"({incomplete_series / n_series * 100:.1f}%)")

```

Stores: 54

Product families: 33

Store x family series: 1782

Expected days per complete series: 1688

Series with at least one missing day: 1782 of 1782 (100.0%)

### 1.2.6 Missing-day distribution across series

```

[ ]: def missing_days_per_series(data, full_range):
    """How many missing calendar days does each store x family series have?
    A distribution across all series, to check whether gaps are a handful of
    days each or something more systemic (a full histogram is plotted in the
    EDA section below)."""
    counts = data.groupby(["store_nbr", "family"], observed=True)["date"].
    ↪nunique()
    missing = len(full_range) - counts
    return missing

missing_per_series = missing_days_per_series(train, full_date_range)
print(missing_per_series.describe())

```

```

count      1782.0
mean         4.0
std          0.0
min          4.0
25%          4.0
50%          4.0
75%          4.0
max          4.0
Name: date, dtype: float64

```

### 1.2.7 Oil price gaps

```
[9]: oil_full = oil.set_index("date").reindex(full_date_range)
missing_oil_pct = oil_full["dcoilwtico"].isnull().mean() * 100
print(f"Missing oil-price days: {missing_oil_pct:.1f}% of the full date range")
```

Missing oil-price days: 31.1% of the full date range

### 1.2.8 Holiday types and the transferred flag

```
[10]: print(holidays["type"].value_counts())
print(f"\nTransferred=True rows: {holidays['transferred'].sum()} "
      f"(these do NOT count as a holiday on their listed date - see Process)")
```

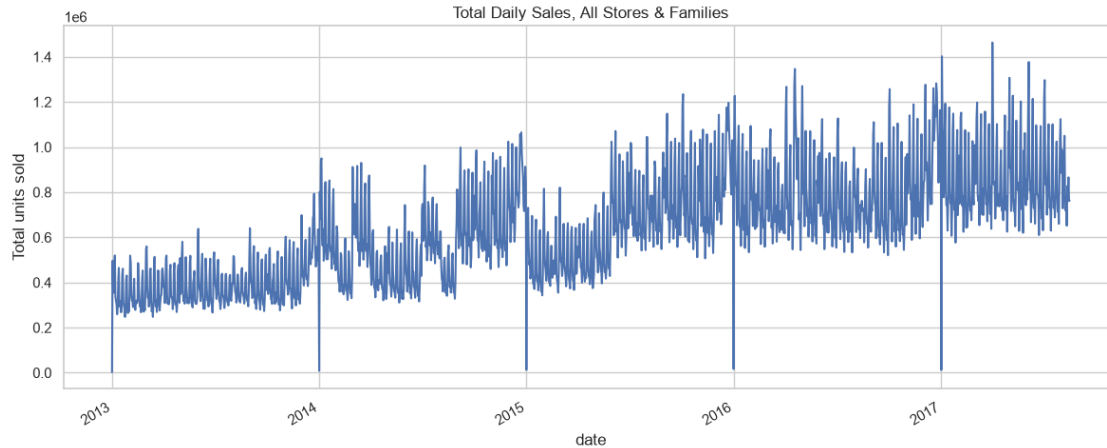
```
type
Holiday      221
Event         56
Additional    51
Transfer      12
Bridge        5
Work Day      5
Name: count, dtype: int64
```

Transferred=True rows: 12 (these do NOT count as a holiday on their listed date - see Process)

## 1.3 Exploratory Data Analysis

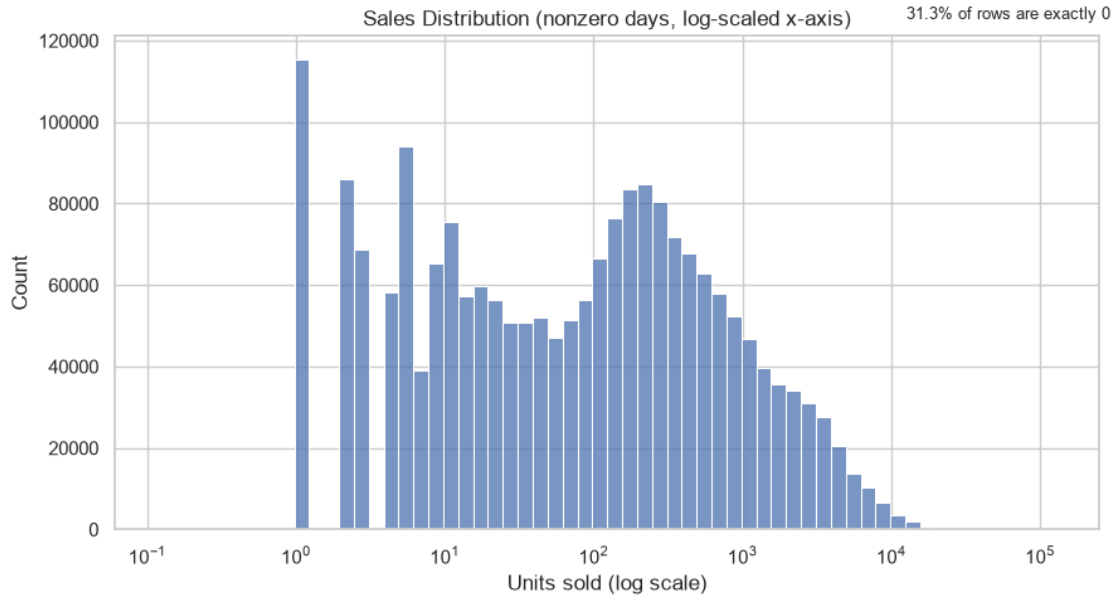
```
[ ]: def plot_total_sales_over_time(data):
    daily_total = data.groupby("date", observed=True)["sales"].sum()
    plt.figure(figsize=(12, 5))
    daily_total.plot(color="#4C72B0")
    plt.title("Total Daily Sales, All Stores & Families")
    plt.ylabel("Total units sold")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/01_total_sales_over_time.png", dpi=150,
        bbox_inches="tight")
    plt.show()

plot_total_sales_over_time(train)
```



```
[ ]: def plot_sales_distribution(data):
    plt.figure(figsize=(9, 5))
    sns.histplot(data["sales"][data["sales"] > 0], bins=60, log_scale=(True,
↪False), color="#4C72B0")
    plt.title("Sales Distribution (nonzero days, log-scaled x-axis)")
    plt.xlabel("Units sold (log scale)")
    zero_pct = (data["sales"] == 0).mean() * 100
    plt.figtext(0.99, 0.95, f"{zero_pct:.1f}% of rows are exactly 0",
↪ha="right", fontsize=9)
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/02_sales_distribution.png", dpi=150,
↪bbox_inches="tight")
    plt.show()
    print(f"Zero-sales rows: {zero_pct:.1f}% - many series are intermittent,
↪which is exactly "
          f"why MAPE alone is a bad metric here (Part 3.3 of the plan).")

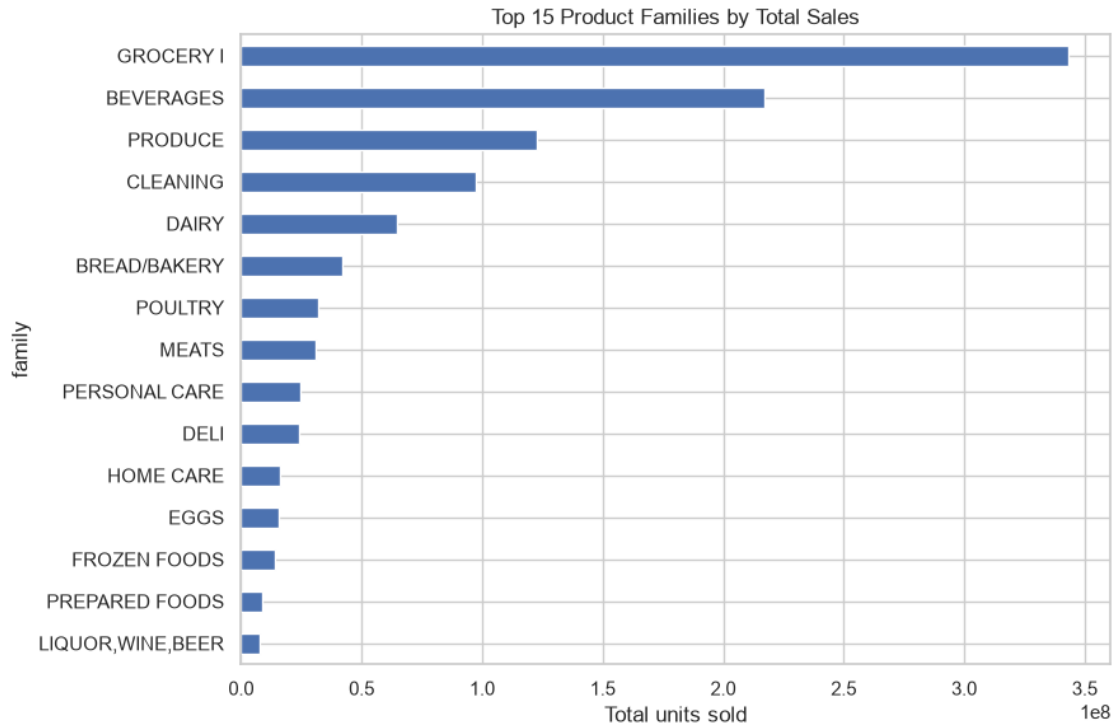
plot_sales_distribution(train)
```



Zero-sales rows: 31.3% - many series are intermittent, which is exactly why MAPE alone is a bad metric here (Part 3.3 of the plan).

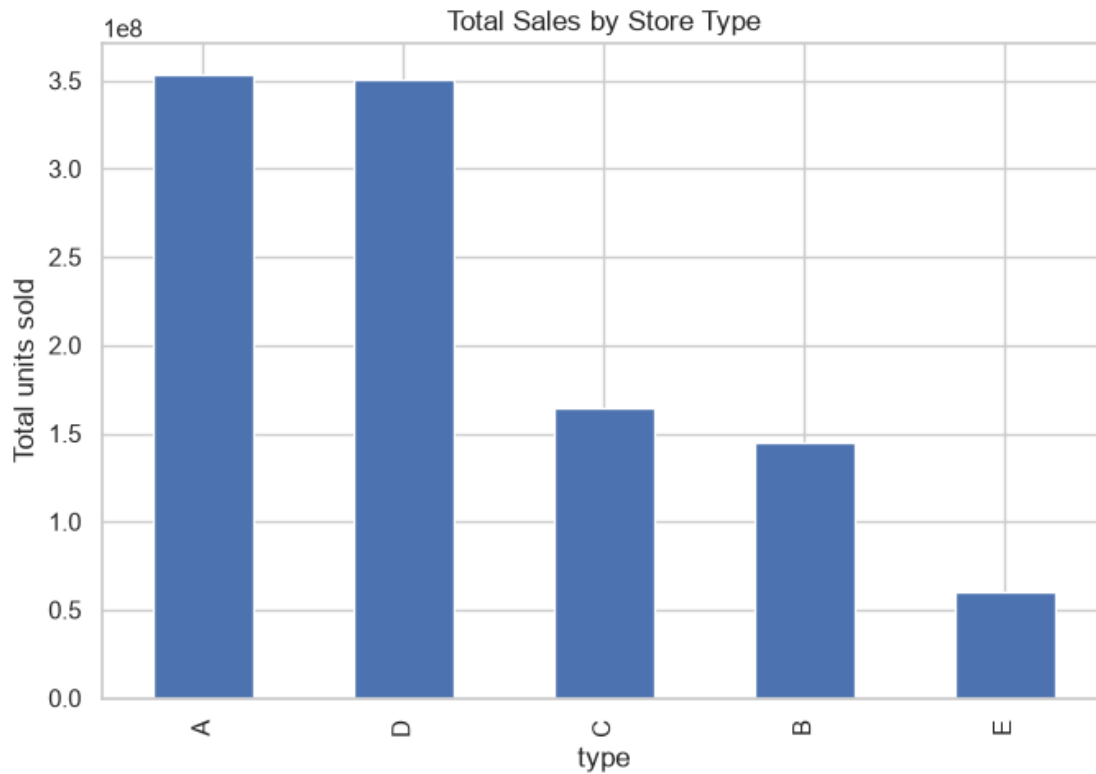
```
[ ]: def plot_sales_by_family(data, top_n=15):
    top_families = data.groupby("family", observed=True)["sales"].sum().
    ↪sort_values(ascending=False).head(top_n)
    plt.figure(figsize=(9, 6))
    top_families.plot(kind="barh", color="#4C72B0")
    plt.gca().invert_yaxis()
    plt.title(f"Top {top_n} Product Families by Total Sales")
    plt.xlabel("Total units sold")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/03_sales_by_family.png", dpi=150,
    ↪bbox_inches="tight")
    plt.show()
    return top_families
```

```
top_families = plot_sales_by_family(train)
```



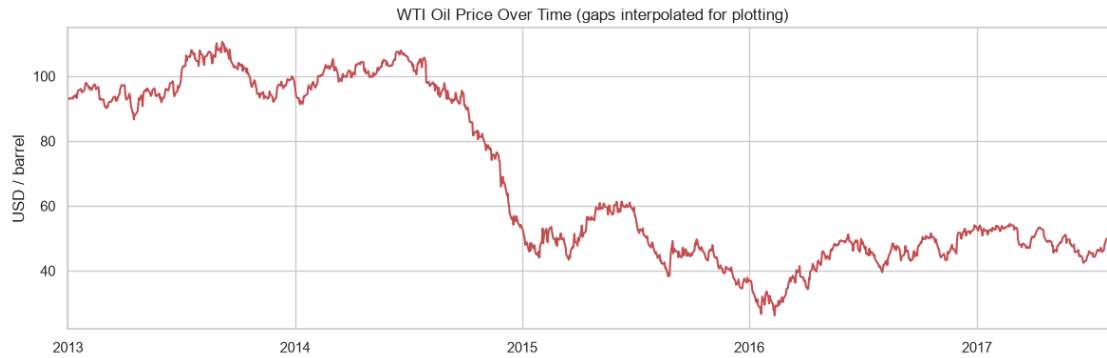
```
[ ]: def plot_sales_by_store_type(data, stores_df):
    merged = data.merge(stores_df[["store_nbr", "type"]], on="store_nbr",
        ↳how="left")
    by_type = merged.groupby("type", observed=True)["sales"].sum().
        ↳sort_values(ascending=False)
    plt.figure(figsize=(7, 5))
    by_type.plot(kind="bar", color="#4C72B0")
    plt.title("Total Sales by Store Type")
    plt.ylabel("Total units sold")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/04_sales_by_store_type.png", dpi=150,
        ↳bbox_inches="tight")
    plt.show()

plot_sales_by_store_type(train, stores)
```



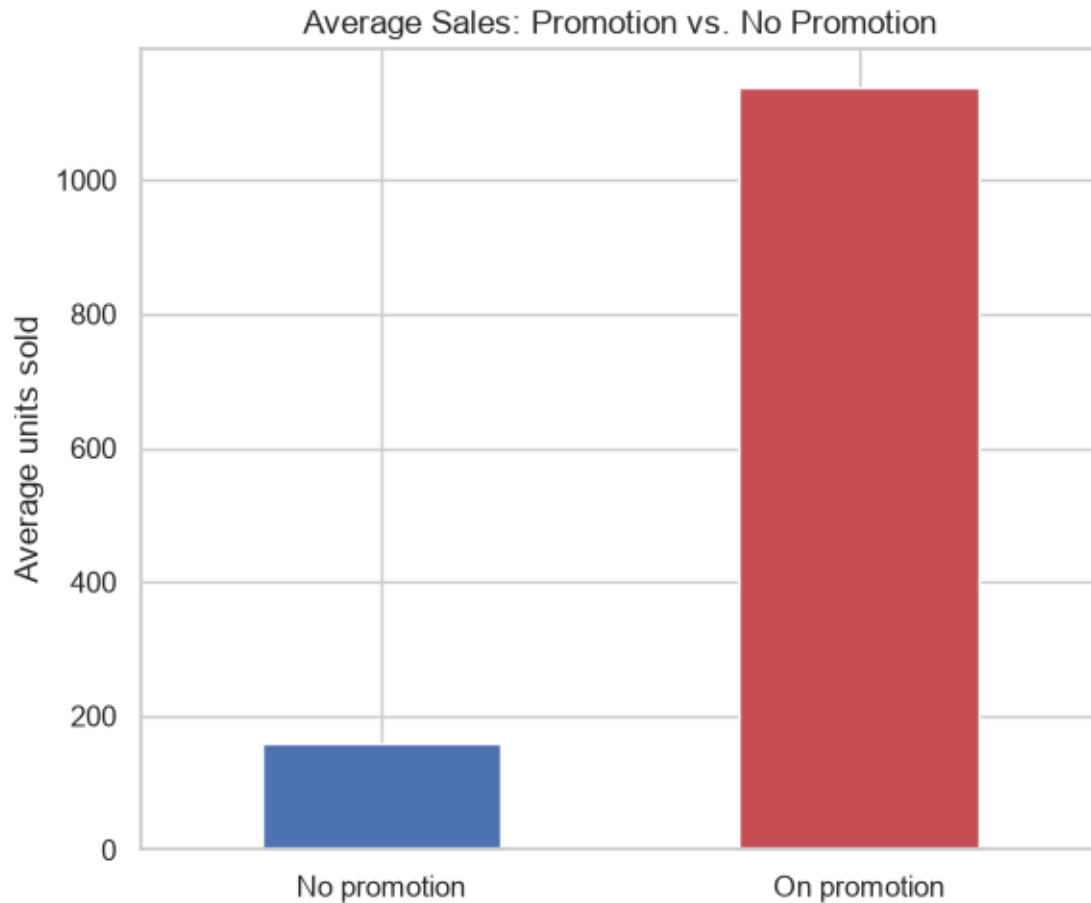
```
[ ]: def plot_oil_price(oil_df, full_range):
    filled = oil_df.set_index("date").reindex(full_range)["dcoilwtico"].
    ↪interpolate()
    plt.figure(figsize=(12, 4))
    filled.plot(color="#C44E52")
    plt.title("WTI Oil Price Over Time (gaps interpolated for plotting)")
    plt.ylabel("USD / barrel")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/05_oil_price_over_time.png", dpi=150,
    ↪bbox_inches="tight")
    plt.show()

plot_oil_price(oil, full_date_range)
```



```
[ ]: def plot_promotion_effect(data):
    has_promo = data["onpromotion"] > 0
    avg_sales = data.groupby(has_promo)["sales"].mean()
    avg_sales.index = ["No promotion", "On promotion"]
    plt.figure(figsize=(6, 5))
    avg_sales.plot(kind="bar", color=["#4C72B0", "#C44E52"])
    plt.title("Average Sales: Promotion vs. No Promotion")
    plt.ylabel("Average units sold")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/06_promotion_effect.png", dpi=150,
    ↳bbox_inches="tight")
    plt.show()
    print(avg_sales)

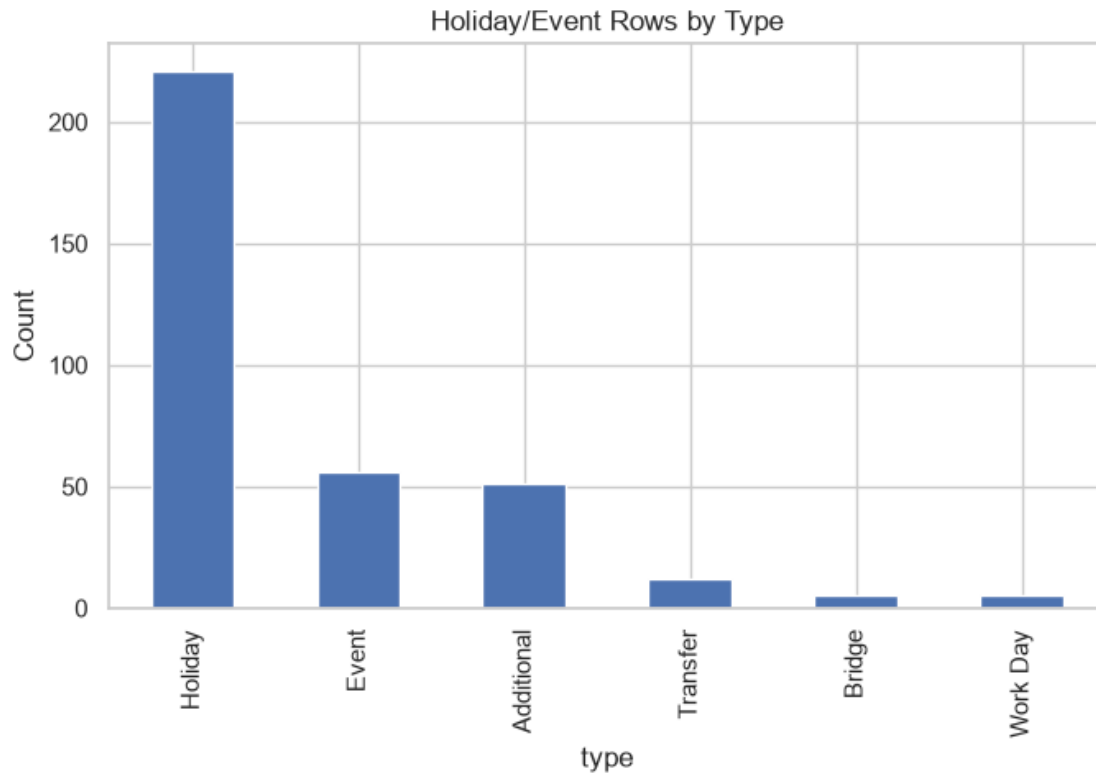
plot_promotion_effect(train)
```



```
No promotion    158.246674
On promotion    1137.693726
Name: sales, dtype: float32
```

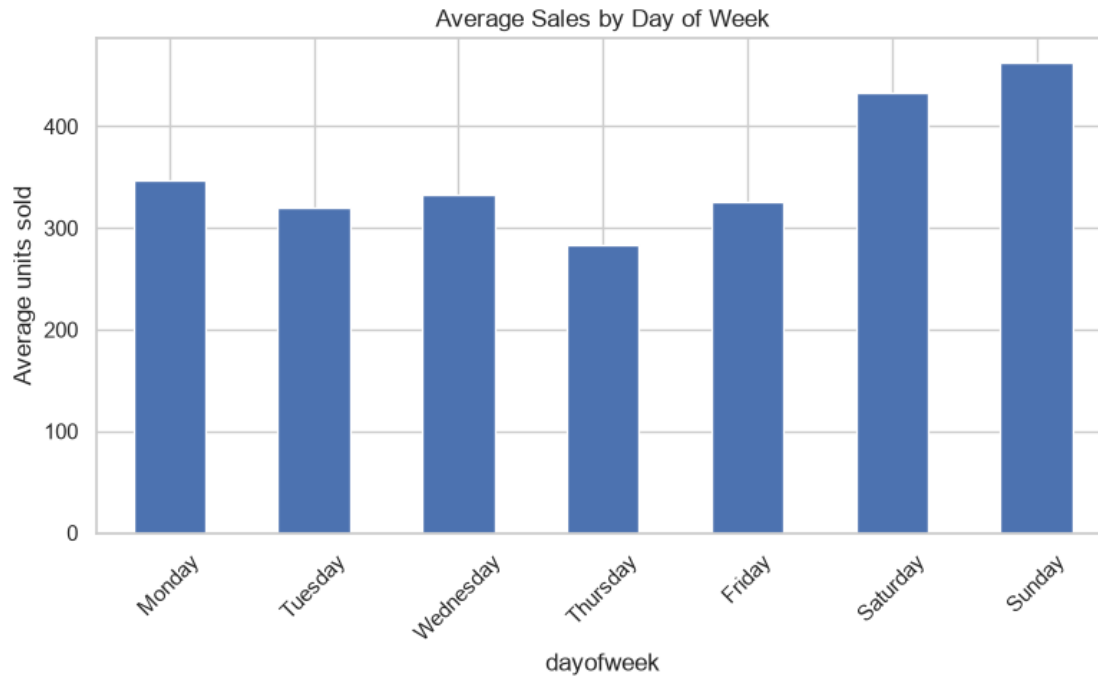
```
[ ]: def plot_holiday_counts(holidays_df):
    plt.figure(figsize=(7, 5))
    holidays_df["type"].value_counts().plot(kind="bar", color="#4C72B0")
    plt.title("Holiday/Event Rows by Type")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/07_holiday_counts.png", dpi=150,
    ↳bbox_inches="tight")
    plt.show()

plot_holiday_counts(holidays)
```



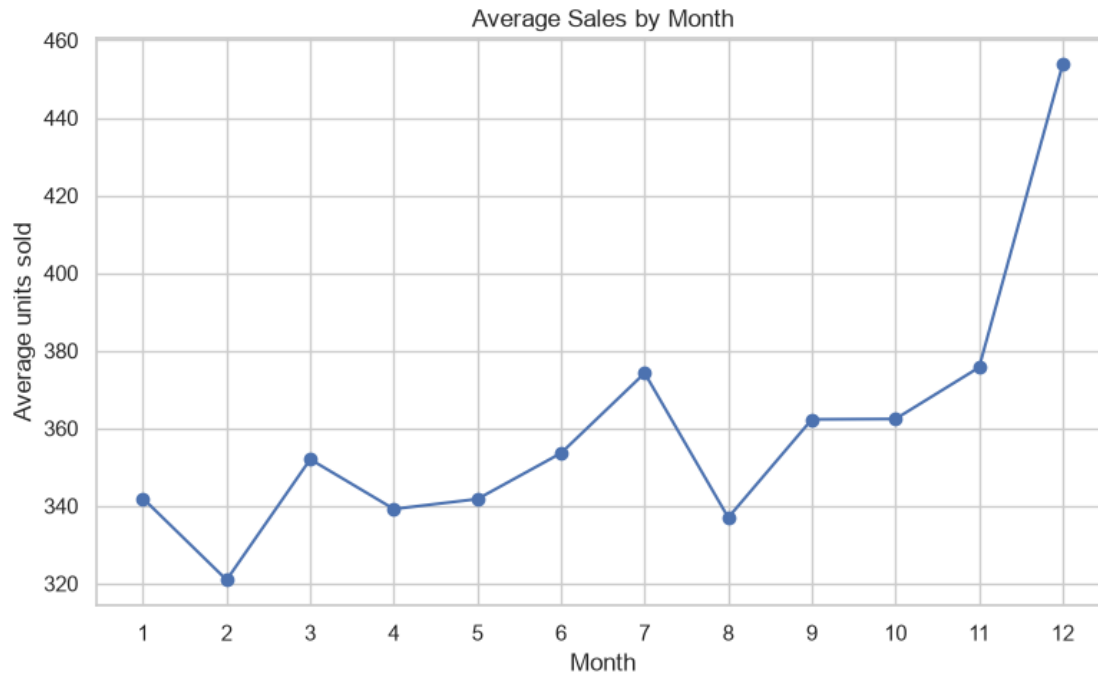
```
[ ]: def plot_dayofweek_seasonality(data):
    temp = data.copy()
    temp["dayofweek"] = temp["date"].dt.day_name()
    order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
    ↪ "Saturday", "Sunday"]
    avg = temp.groupby("dayofweek", observed=True)["sales"].mean().
    ↪ reindex(order)
    plt.figure(figsize=(8, 5))
    avg.plot(kind="bar", color="#4C72B0")
    plt.title("Average Sales by Day of Week")
    plt.ylabel("Average units sold")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/08_dayofweek_seasonality.png", dpi=150,
    ↪ bbox_inches="tight")
    plt.show()

plot_dayofweek_seasonality(train)
```



```
[ ]: def plot_monthly_seasonality(data):
    temp = data.copy()
    temp["month"] = temp["date"].dt.month
    avg = temp.groupby("month", observed=True)["sales"].mean()
    plt.figure(figsize=(8, 5))
    avg.plot(kind="line", marker="o", color="#4C72B0")
    plt.title("Average Sales by Month")
    plt.xlabel("Month")
    plt.ylabel("Average units sold")
    plt.xticks(range(1, 13))
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/09_monthly_seasonality.png", dpi=150,
    ↳bbox_inches="tight")
    plt.show()

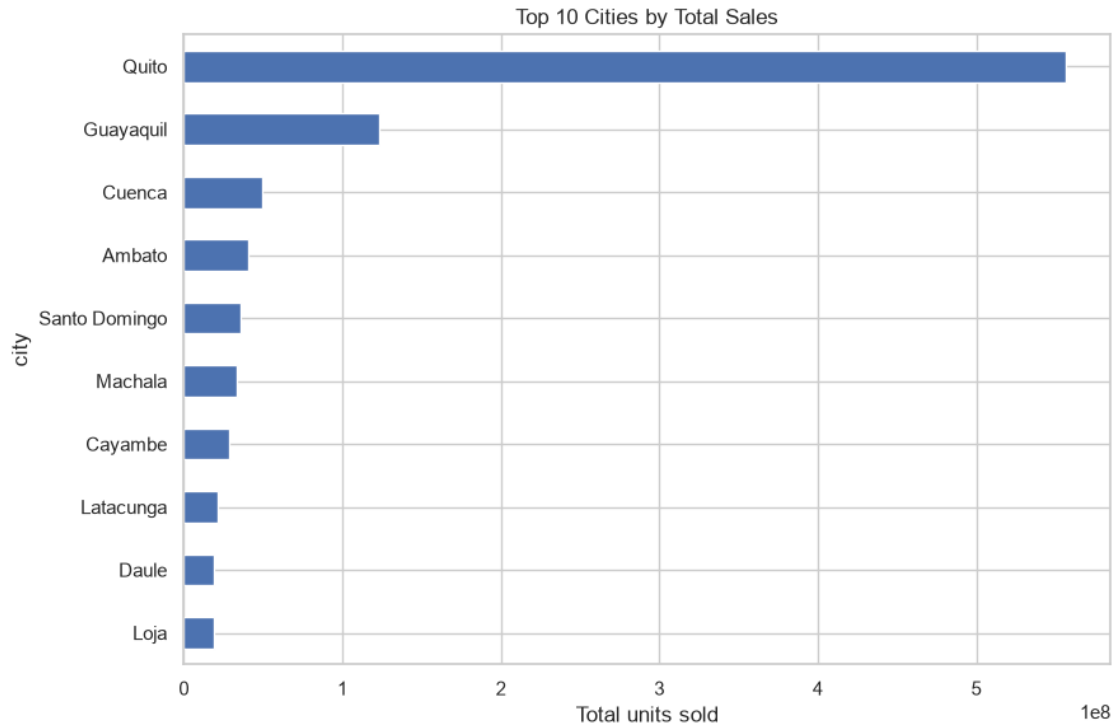
plot_monthly_seasonality(train)
```



### 1.3.1 Sales by city

```
[ ]: def plot_sales_by_city(data, stores_df, top_n=10):
    merged = data.merge(stores_df[["store_nbr", "city"]], on="store_nbr",
        ↳how="left")
    by_city = merged.groupby("city", observed=True)["sales"].sum().
        ↳sort_values(ascending=False).head(top_n)
    plt.figure(figsize=(9, 6))
    by_city.plot(kind="barh", color="#4C72B0")
    plt.gca().invert_yaxis()
    plt.title(f"Top {top_n} Cities by Total Sales")
    plt.xlabel("Total units sold")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/10_sales_by_city.png", dpi=150,
        ↳bbox_inches="tight")
    plt.show()

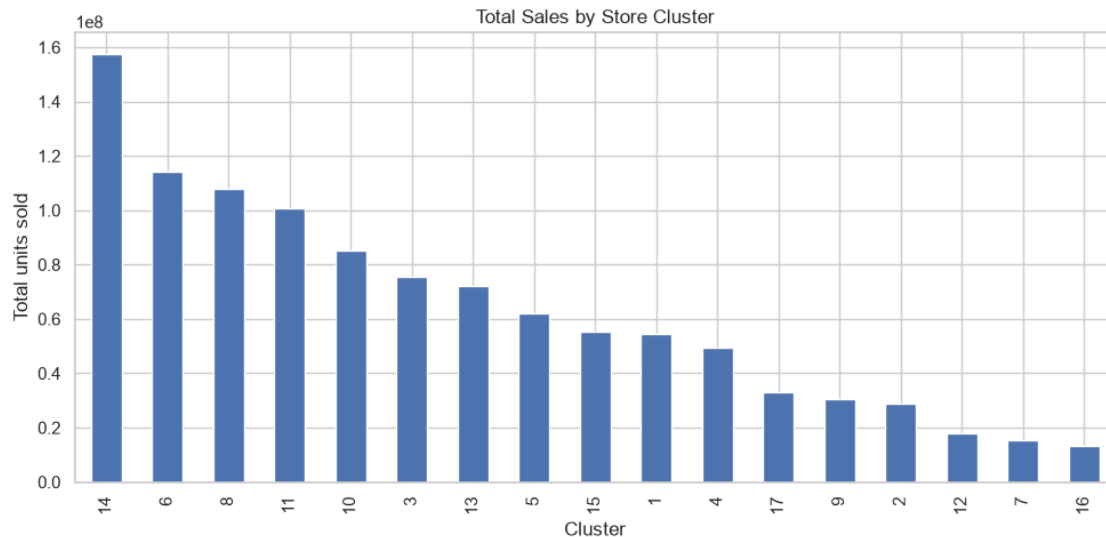
plot_sales_by_city(train, stores)
```



### 1.3.2 Sales by store cluster

```
[ ]: def plot_sales_by_cluster(data, stores_df):
    merged = data.merge(stores_df[["store_nbr", "cluster"]], on="store_nbr",
    ↪how="left")
    by_cluster = merged.groupby("cluster", observed=True)["sales"].sum().
    ↪sort_values(ascending=False)
    plt.figure(figsize=(10, 5))
    by_cluster.plot(kind="bar", color="#4C72B0")
    plt.title("Total Sales by Store Cluster")
    plt.xlabel("Cluster")
    plt.ylabel("Total units sold")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/11_sales_by_cluster.png", dpi=150,
    ↪bbox_inches="tight")
    plt.show()

plot_sales_by_cluster(train, stores)
```

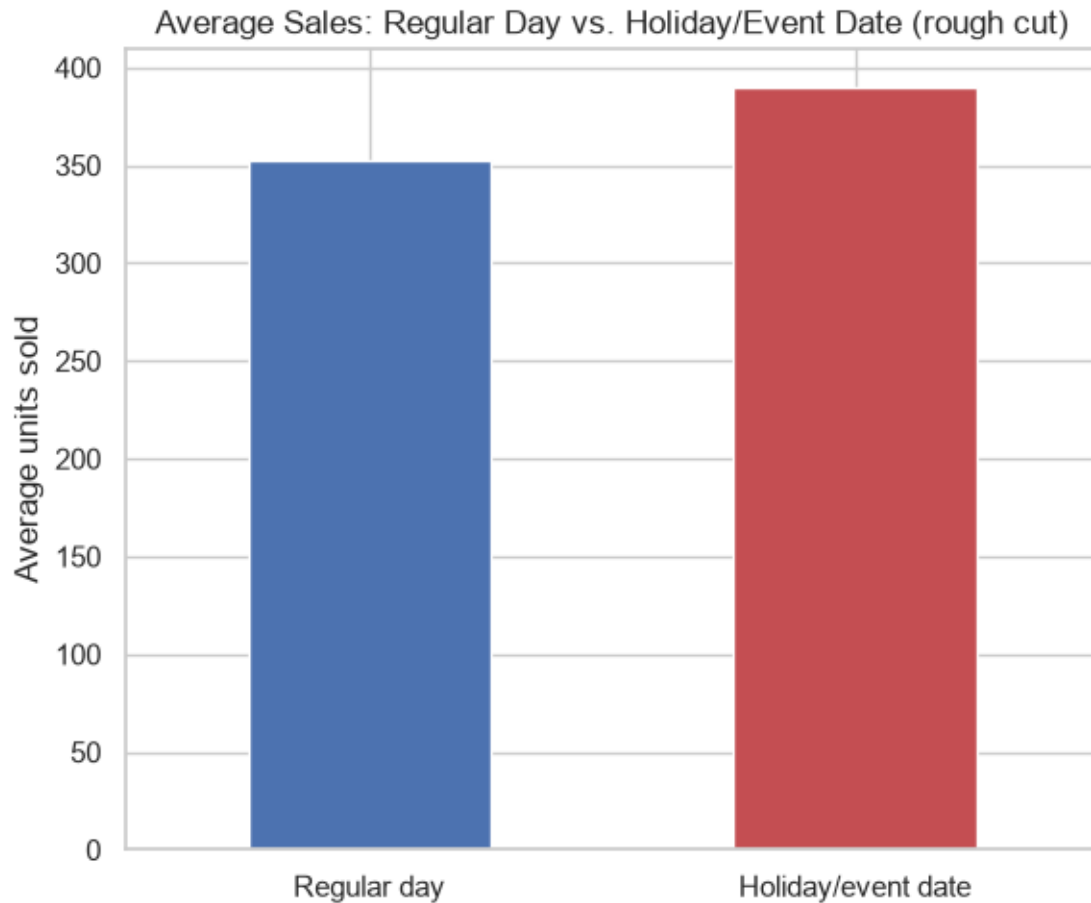


### 1.3.3 Holiday/event effect on sales (descriptive cut)

A quick descriptive comparison only - this uses every date that appears anywhere in `holidays_events.csv`, not the leakage-safe transferred-flag resolution built in Process below. Treat this as an EDA sanity check, not a modeling feature.

```
[ ]: def plot_holiday_effect(data, holidays_df):
    any_holiday_dates = set(holidays_df["date"])
    temp = data.copy()
    temp["is_any_holiday_date"] = temp["date"].isin(any_holiday_dates)
    avg = temp.groupby("is_any_holiday_date")["sales"].mean()
    avg.index = ["Regular day", "Holiday/event date"]
    plt.figure(figsize=(6, 5))
    avg.plot(kind="bar", color=["#4C72B0", "#C44E52"])
    plt.title("Average Sales: Regular Day vs. Holiday/Event Date (rough cut)")
    plt.ylabel("Average units sold")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/12_holiday_effect.png", dpi=150,
    bbox_inches="tight")
    plt.show()
    print(avg)

plot_holiday_effect(train, holidays)
```



```
Regular day          352.159180
Holiday/event date    389.692139
Name: sales, dtype: float32
```

### 1.3.4 Transactions vs. sales

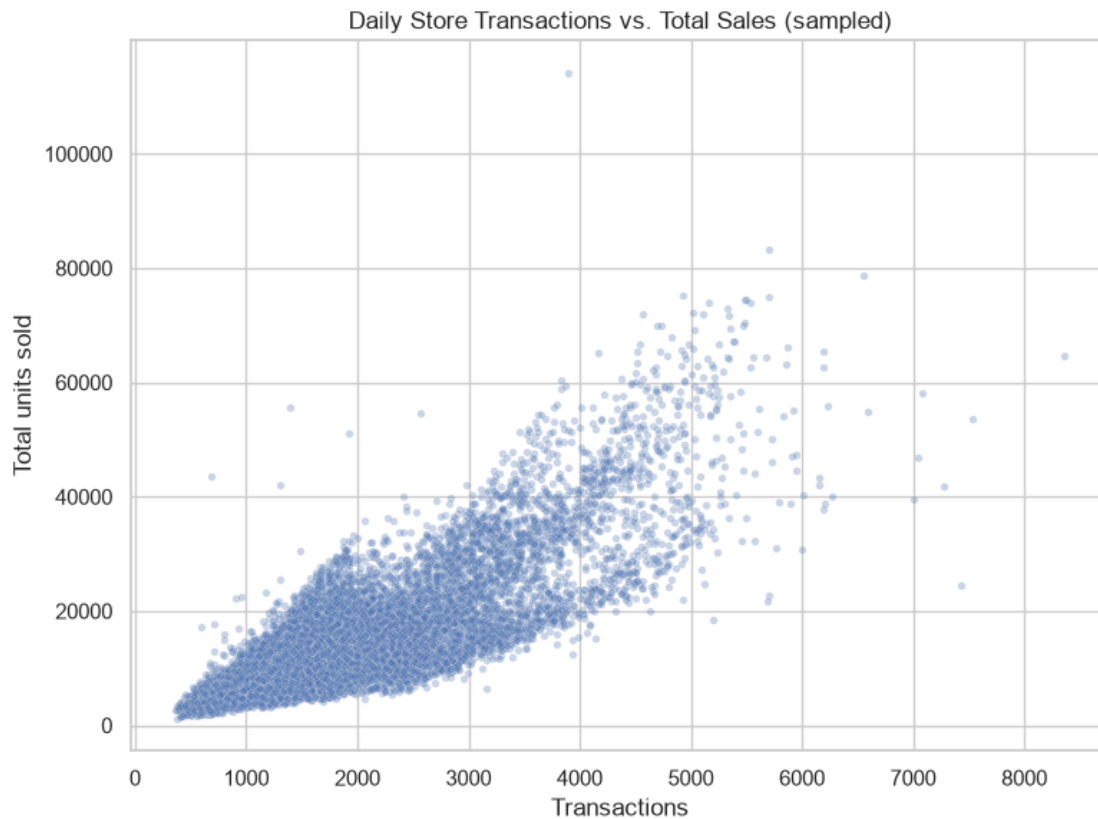
```
[ ]: def plot_transactions_vs_sales(data, transactions_df, sample_size=20_000):
    daily_sales = data.groupby(["date", "store_nbr"], observed=True)["sales"].
    ↪sum().reset_index()
    merged = daily_sales.merge(transactions_df, on=["date", "store_nbr"],
    ↪how="inner")
    if len(merged) > sample_size:
        merged = merged.sample(sample_size, random_state=42)
    plt.figure(figsize=(8, 6))
    sns.scatterplot(data=merged, x="transactions", y="sales", alpha=0.3, s=15,
    ↪color="#4C72B0")
    plt.title("Daily Store Transactions vs. Total Sales (sampled)")
    plt.xlabel("Transactions")
```

```

plt.ylabel("Total units sold")
plt.tight_layout()
plt.savefig(f"{ASSETS_DIR}/13_transactions_vs_sales_scatter.png", dpi=150,
bbox_inches="tight")
plt.show()
corr = merged["transactions"].corr(merged["sales"])
print(f"Correlation (transactions, sales): {corr:.3f}")

plot_transactions_vs_sales(train, transactions)

```



Correlation (transactions, sales): 0.834

### 1.3.5 Correlation snapshot (raw daily aggregates, before any feature engineering)

```

[ ]: def plot_raw_correlation_heatmap(data, transactions_df, oil_df):
    daily = data.groupby(["date", "store_nbr"], observed=True).agg(
        sales=("sales", "sum"), onpromotion=("onpromotion", "sum")
    ).reset_index()
    daily = daily.merge(transactions_df, on=["date", "store_nbr"], how="left")
    daily = daily.merge(oil_df, on="date", how="left")

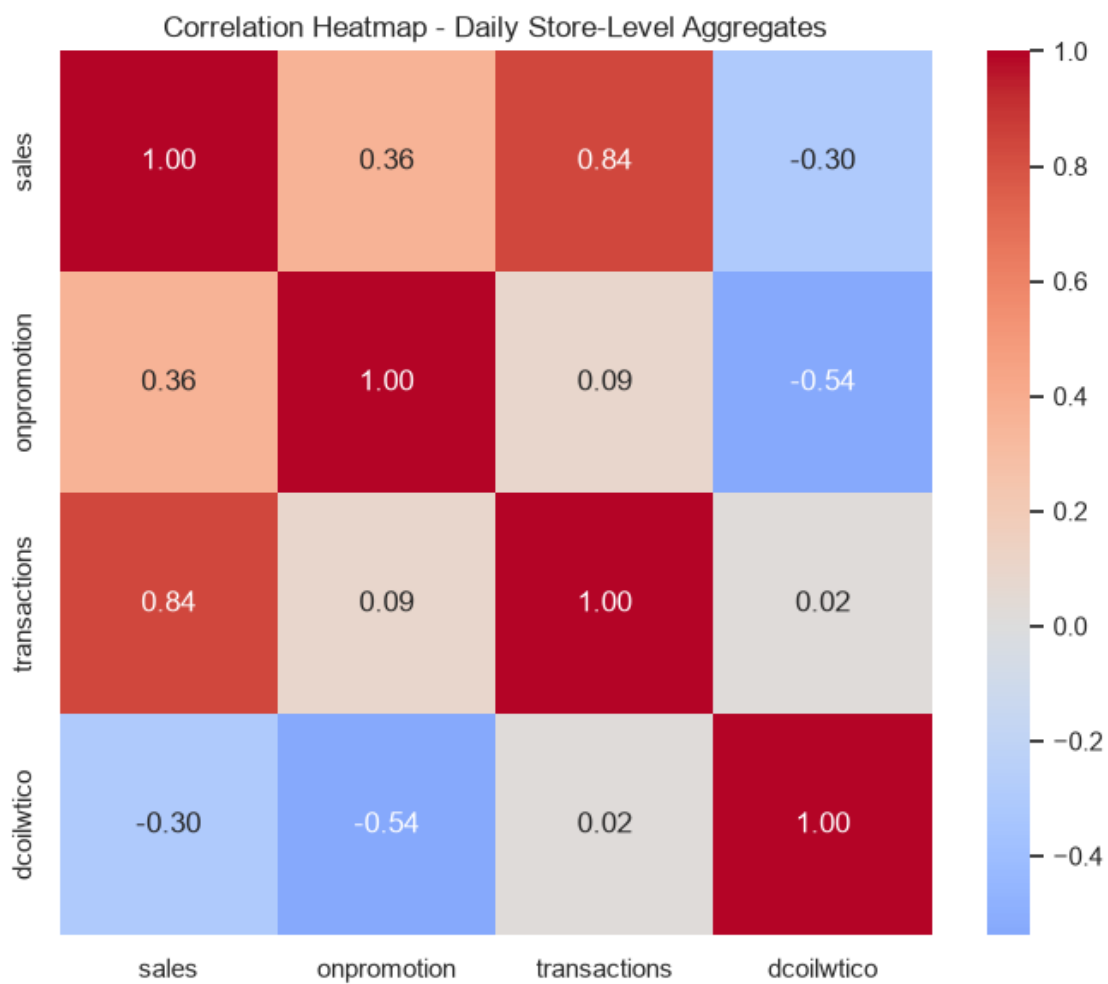
```

```

corr_cols = ["sales", "onpromotion", "transactions", "dcoilwtico"]
plt.figure(figsize=(7, 6))
sns.heatmap(daily[corr_cols].corr(), cmap="coolwarm", center=0, annot=True,
fmt=".2f")
plt.title("Correlation Heatmap - Daily Store-Level Aggregates")
plt.tight_layout()
plt.savefig(f"{ASSETS_DIR}/14_raw_correlation_heatmap.png", dpi=150,
bbox_inches="tight")
plt.show()

```

```
plot_raw_correlation_heatmap(train, transactions, oil)
```



### 1.3.6 Series completeness, visualized

```
[ ]: def plot_missing_days_distribution(missing_series):  
    plt.figure(figsize=(9, 5))  
    sns.histplot(missing_series, bins=30, color="#4C72B0")  
    plt.title("Distribution of Missing Calendar Days per Store x Family Series")  
    plt.xlabel("Missing days out of the full date range")  
    plt.ylabel("Number of series")  
    plt.tight_layout()  
    plt.savefig(f"{ASSETS_DIR}/15_missing_days_distribution.png", dpi=150,  
↳ bbox_inches="tight")  
    plt.show()  
  
plot_missing_days_distribution(missing_per_series)
```



## 1.4 3. Process - Clean and Engineer Features

### 1.4.1 Resolving the transferred-holiday flag

This is the documented gotcha in this dataset: a holiday marked `transferred=True` did NOT happen on its listed date - it moved to a different date, which shows up as its own row with `type == "Transfer"`. `type == "Work Day"` means a normally non-working day was turned INTO a working day (compensating for a bridge elsewhere), so it should not be flagged as a holiday either.

```
[26]: def build_holiday_flag(holidays_df):  
    """Returns the set of dates that are actually non-working/holiday days,  
    with the transferred-flag logic resolved correctly."""
```

```

df = holidays_df.copy()

# Rows where transferred=True did not happen on their listed date - drop
↳ them here.
observed = df[~df["transferred"]]

# Work Day rows cancel out a bridge elsewhere - they are NOT holidays.
# Everything else (Holiday, Additional, Bridge, Transfer, Event) is treated
↳ as non-working.
holiday_dates = set(observed.loc[observed["type"] != "Work Day", "date"])
return holiday_dates

# Simplification: only National-level holidays are applied uniformly to every
↳ store here.
# Regional/Local holidays would need matching each store's city/state from
↳ stores.csv -
# a reasonable next step, left out here to keep the first pass tractable (see
↳ Limitations).
national_holidays = holidays[holidays["locale"] == "National"]
holiday_dates = build_holiday_flag(national_holidays)
print(f"Resolved {len(holiday_dates)} national holiday/non-working dates.")

```

Resolved 155 national holiday/non-working dates.

### 1.4.2 Merging everything onto the sales table

```

[ ]: data = train.merge(stores, on="store_nbr", how="left")

oil_filled = oil.set_index("date").reindex(full_date_range)["dcoilwtico"].
↳ interpolate().ffill().bfill()
oil_filled = oil_filled.rename("oil_price").reset_index().
↳ rename(columns={"index": "date"})
data = data.merge(oil_filled, on="date", how="left")

data["is_holiday"] = data["date"].isin(holiday_dates).astype("int8")

data = data.merge(transactions, on=["date", "store_nbr"], how="left")
data["transactions"] = data.groupby("store_nbr", observed=True)["transactions"].
↳ transform(
    lambda s: s.fillna(s.median())
)

print(f"Merged shape: {data.shape}")
data.head()

```

Merged shape: (3000888, 13)

```
[ ]:  id      date  store_nbr      family  sales  onpromotion  city      state \
0    0 2013-01-01          1  AUTOMOTIVE    0.0           0  Quito  Pichincha
1    1 2013-01-01          1   BABY CARE    0.0           0  Quito  Pichincha
2    2 2013-01-01          1     BEAUTY    0.0           0  Quito  Pichincha
3    3 2013-01-01          1  BEVERAGES    0.0           0  Quito  Pichincha
4    4 2013-01-01          1     BOOKS     0.0           0  Quito  Pichincha

      type  cluster  oil_price  is_holiday  transactions
0      D        13  93.139999           1           1746.0
1      D        13  93.139999           1           1746.0
2      D        13  93.139999           1           1746.0
3      D        13  93.139999           1           1746.0
4      D        13  93.139999           1           1746.0
```

### 1.4.3 Lag, rolling, and calendar features (computed per store x family to avoid leaking across series)

```
[ ]: data = data.sort_values(["store_nbr", "family", "date"])
grp = data.groupby(["store_nbr", "family"], observed=True)["sales"]

for lag in [7, 14, 28]:
    data[f"sales_lag_{lag}"] = grp.shift(lag)

# Rolling stats computed on already-shifted (lag-1) sales, so today's own value
↳ never leaks in.
shifted = grp.shift(1)
data["rolling_mean_7"] = shifted.groupby([data["store_nbr"], data["family"]]).
↳ transform(
    lambda s: s.rolling(7, min_periods=1).mean()
)
data["rolling_mean_28"] = shifted.groupby([data["store_nbr"], data["family"]]).
↳ transform(
    lambda s: s.rolling(28, min_periods=1).mean()
)
data["rolling_std_7"] = shifted.groupby([data["store_nbr"], data["family"]]).
↳ transform(
    lambda s: s.rolling(7, min_periods=1).std()
)

data["dayofweek"] = data["date"].dt.dayofweek
data["month"] = data["date"].dt.month
data["day"] = data["date"].dt.day
data["is_weekend"] = data["dayofweek"].isin([5, 6]).astype("int8")
data["any_promotion"] = (data["onpromotion"] > 0).astype("int8")
data["oil_price_change"] = data["oil_price"].diff().fillna(0)
```

```
# Drop the warm-up rows at the start of each series where lag_28 isn't
↳available yet.
before = len(data)
data = data.dropna(subset=["sales_lag_28"]).reset_index(drop=True)
print(f"Dropped {before - len(data):,} warm-up rows; {len(data):,} rows remain.
↳")
```

Dropped 49,896 warm-up rows; 2,950,992 rows remain.

#### 1.4.4 One-hot encoding + chronological split (never a random shuffle for time series)

```
[ ]: data = pd.get_dummies(data, columns=["family", "type"], prefix=["family",
↳"store_type"], dtype="int8")

feature_cols_base = [
    "onpromotion", "any_promotion", "is_holiday", "cluster",
    "sales_lag_7", "sales_lag_14", "sales_lag_28",
    "rolling_mean_7", "rolling_mean_28", "rolling_std_7",
    "dayofweek", "month", "day", "is_weekend", "transactions",
] + [c for c in data.columns if c.startswith("family_") or c.
↳startswith("store_type_")]

feature_cols_with_oil = feature_cols_base + ["oil_price", "oil_price_change"]

TEST_DAYS = 15
max_date = data["date"].max()
test_start = max_date - pd.Timedelta(days=TEST_DAYS - 1)
val_start = test_start - pd.Timedelta(days=TEST_DAYS)

train_df = data[data["date"] < val_start]
val_df = data[(data["date"] >= val_start) & (data["date"] < test_start)]
test_df = data[data["date"] >= test_start]

for name, d in [("Train", train_df), ("Validation", val_df), ("Test", test_df)]:
    print(f"{name}: {d['date'].min().date()} to {d['date'].max().date()},
↳{len(d):,} rows")
```

Train: 2013-01-29 to 2017-07-16, 2,897,532 rows  
 Validation: 2017-07-17 to 2017-07-31, 26,730 rows  
 Test: 2017-08-01 to 2017-08-15, 26,730 rows

### 1.5 4. Analyze - Model Building and Comparison

Following the experiment matrix from the plan: naive baselines, ARIMA and Prophet fit per-series on a handful of representative high-volume series, then global Random Forest / XGBoost / LSTM models trained across all series at once.

```

[]: results = []

def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

def mape(y_true, y_pred):
    mask = y_true != 0
    if mask.sum() == 0:
        return np.nan
    return float(np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100)

def wape(y_true, y_pred):
    denom = np.sum(np.abs(y_true))
    if denom == 0:
        return np.nan
    return float(np.sum(np.abs(y_true - y_pred)) / denom * 100)

def forecast_bias(y_true, y_pred):
    denom = np.sum(y_true)
    if denom == 0:
        return np.nan
    return float((np.sum(y_pred) - denom) / denom * 100)

def evaluate_forecast(name, y_true, y_pred):
    y_true = np.asarray(y_true, dtype="float64")
    y_pred = np.asarray(y_pred, dtype="float64")
    metrics = {
        "Experiment": name,
        "RMSE": rmse(y_true, y_pred),
        "MAPE_%": mape(y_true, y_pred),
        "WAPE_%": wape(y_true, y_pred),
        "Bias_%": forecast_bias(y_true, y_pred),
    }
    results.append(metrics)
    print(f"--- {name} ---")
    print(f"RMSE: {metrics['RMSE']:.3f}  MAPE: {metrics['MAPE_%']:.2f}%  "
          f"WAPE: {metrics['WAPE_%']:.2f}%  Bias: {metrics['Bias_%']:.2f}%")
    return metrics

```

### 1.5.1 Picking representative series

ARIMA and Prophet are fit one series at a time, which doesn't scale to ~1,782 series in a first pass. Following the plan's recommended role, they're fit on the top 5 highest-volume store x family series, and every model (including the global ones) is compared on this same subset for a fair, apples-to-apples table - the global models are *also* evaluated on the full test set separately, further down.

```
[ ]: series_totals = train.groupby(["store_nbr", "family"], observed=True)["sales"].
    ↪sum().sort_values(ascending=False)
representative_series = list(series_totals.head(5).index)
print("Representative series (store_nbr, family):")
for s in representative_series:
    print(f" {s} (total sales: {series_totals[s]:,.0f})")
```

```
Representative series (store_nbr, family):
(44, 'GROCERY I') (total sales: 16,386,055)
(45, 'GROCERY I') (total sales: 16,349,751)
(47, 'GROCERY I') (total sales: 15,514,528)
(46, 'GROCERY I') (total sales: 14,342,262)
(44, 'BEVERAGES') (total sales: 13,417,859)
```

### 1.5.2 Experiment 1 - Naive & seasonal-naive baseline

```
[ ]: naive_true, naive_pred, snaive_pred = [], [], []

for store_nbr, family in representative_series:
    series = data[(data["store_nbr"] == store_nbr) & (data[f"family_{family}"]_
    ↪== 1)].sort_values("date") \
        if f"family_{family}" in data.columns else None
    # Fall back to the pre-dummy frame for a clean single-series slice.
    series = train[(train["store_nbr"] == store_nbr) & (train["family"] ==_
    ↪family)].sort_values("date")
    series = series.set_index("date")["sales"]

    test_slice = series.loc[test_start:max_date]
    naive_forecast = series.shift(1).loc[test_start:max_date] #_
    ↪yesterday's value
    seasonal_naive_forecast = series.shift(7).loc[test_start:max_date] # same_
    ↪weekday last week

    naive_true.extend(test_slice.values)
    naive_pred.extend(naive_forecast.values)
    snaive_pred.extend(seasonal_naive_forecast.values)

naive_true = np.array(naive_true)
naive_pred = np.nan_to_num(np.array(naive_pred))
snaive_pred = np.nan_to_num(np.array(snaive_pred))

evaluate_forecast("1a. Naive (yesterday's value)", naive_true, naive_pred)
evaluate_forecast("1b. Seasonal-naive (same weekday last week)", naive_true,_
    ↪snaive_pred)
```

```
--- 1a. Naive (yesterday's value) ---
RMSE: 2063.516 MAPE: 16.22% WAPE: 15.77% Bias: +1.93%
--- 1b. Seasonal-naive (same weekday last week) ---
```

RMSE: 2503.252 MAPE: 21.75% WAPE: 21.14% Bias: +8.66%

```
[ ]: {'Experiment': '1b. Seasonal-naive (same weekday last week)',  
      'RMSE': 2503.2524443211873,  
      'MAPE_%': 21.751653873736483,  
      'WAPE_%': 21.14225220212001,  
      'Bias_%': 8.660812443165623}
```

### 1.5.3 Experiment 2 - ARIMA (SARIMAX with weekly seasonality) per representative series

```
[ ]: arima_true, arima_pred = [], []  
  
for store_nbr, family in representative_series:  
    series = train[(train["store_nbr"] == store_nbr) & (train["family"] ==  
    ↪family)].sort_values("date")  
    series = series.set_index("date")["sales"].asfreq("D").fillna(0)  
  
    train_part = series.loc[:val_start - pd.Timedelta(days=1)]  
    test_part = series.loc[test_start:max_date]  
  
    # Plain ARIMA has no seasonal term, so a seasonal_order handles the weekly,  
    ↪pattern  
    # instead of manually de-seasonalizing first - the "seasonal ARIMA"  
    ↪(SARIMA) approach.  
    model = SARIMAX(  
        train_part, order=(1, 1, 1), seasonal_order=(1, 1, 1, 7),  
        enforce_stationarity=False, enforce_invertibility=False,  
    )  
    fitted = model.fit(dispatch=False)  
    forecast = fitted.forecast(steps=len(test_part))  
  
    arima_true.extend(test_part.values)  
    arima_pred.extend(forecast.values)  
  
evaluate_forecast("2. ARIMA (SARIMAX, weekly seasonal)", np.array(arima_true),  
    ↪np.array(arima_pred))
```

--- 2. ARIMA (SARIMAX, weekly seasonal) ---

RMSE: 3227.971 MAPE: 27.82% WAPE: 25.92% Bias: +12.26%

```
[ ]: {'Experiment': '2. ARIMA (SARIMAX, weekly seasonal)',  
      'RMSE': 3227.9710422883704,  
      'MAPE_%': 27.821093651485825,  
      'WAPE_%': 25.9190339621162,  
      'Bias_%': 12.262679985248594}
```

### 1.5.4 Experiment 3 - Prophet (with holiday + promotion regressors) per representative series

```
[ ]: prophet_holidays_df = pd.DataFrame({"ds": sorted(holiday_dates), "holiday":  
    ↪ "national_holiday"})  
  
prophet_true, prophet_pred = [], []  
  
for store_nbr, family in representative_series:  
    series = train[(train["store_nbr"] == store_nbr) & (train["family"] ==  
    ↪ family)].sort_values("date")  
    prophet_df = series.rename(columns={"date": "ds", "sales": "y"})[["ds",  
    ↪ "y", "onpromotion"]]  
  
    train_part = prophet_df[prophet_df["ds"] < val_start]  
    test_part = prophet_df[prophet_df["ds"] >= test_start]  
  
    model = Prophet(holidays=prophet_holidays_df, weekly_seasonality=True,  
    ↪ yearly_seasonality=True)  
    model.add_regressor("onpromotion")  
    model.fit(train_part)  
  
    forecast = model.predict(test_part[["ds", "onpromotion"]])  
  
    prophet_true.extend(test_part["y"].values)  
    prophet_pred.extend(forecast["yhat"].values)  
  
evaluate_forecast("3. Prophet (holiday + promotion regressors)", np.  
    ↪ array(prophet_true), np.array(prophet_pred))
```

```
18:02:01 - cmdstanpy - INFO - Chain [1] start processing  
18:02:02 - cmdstanpy - INFO - Chain [1] done processing  
18:02:02 - cmdstanpy - INFO - Chain [1] start processing  
18:02:03 - cmdstanpy - INFO - Chain [1] done processing  
18:02:03 - cmdstanpy - INFO - Chain [1] start processing  
18:02:03 - cmdstanpy - INFO - Chain [1] done processing  
18:02:04 - cmdstanpy - INFO - Chain [1] start processing  
18:02:04 - cmdstanpy - INFO - Chain [1] done processing  
18:02:05 - cmdstanpy - INFO - Chain [1] start processing  
18:02:05 - cmdstanpy - INFO - Chain [1] done processing
```

--- 3. Prophet (holiday + promotion regressors) ---

RMSE: 2031.169 MAPE: 17.07% WAPE: 16.49% Bias: +9.56%

```
[ ]: {'Experiment': '3. Prophet (holiday + promotion regressors)',  
    'RMSE': 2031.1694183254174,  
    'MAPE_%': 17.073822863981484,  
    'WAPE_%': 16.49278997512517,
```

```
'Bias_%': 9.561785627151725}
```

### 1.5.5 Experiments 4-6 - Global Random Forest & XGBoost

```
[ ]: X_train_full = train_df[feature_cols_base]
y_train_full = train_df["sales"]
X_test_full = test_df[feature_cols_base]
y_test_full = test_df["sales"]

rf = RandomForestRegressor(n_estimators=300, max_depth=12,
    ↪random_state=RANDOM_STATE, n_jobs=-1)
rf.fit(X_train_full, y_train_full)
rf_pred_full = rf.predict(X_test_full)

evaluate_forecast("4. Random Forest (global, lags+calendar+promo+holiday)",
    ↪y_test_full.values, rf_pred_full)
```

```
--- 4. Random Forest (global, lags+calendar+promo+holiday) ---
RMSE: 229.122 MAPE: 42.91% WAPE: 14.83% Bias: +4.08%
```

```
[ ]: {'Experiment': '4. Random Forest (global, lags+calendar+promo+holiday)',
      'RMSE': 229.12199526295467,
      'MAPE_%': 42.912381237747574,
      'WAPE_%': 14.834577519014289,
      'Bias_%': 4.0815872416996}
```

```
[ ]: xgb_model = xgb.XGBRegressor(
    n_estimators=400, max_depth=8, learning_rate=0.05,
    random_state=RANDOM_STATE, n_jobs=-1,
)
xgb_model.fit(X_train_full, y_train_full)
xgb_pred_full = xgb_model.predict(X_test_full)

evaluate_forecast("5. XGBoost (global, lags+calendar+promo+holiday)",
    ↪y_test_full.values, xgb_pred_full)
```

```
--- 5. XGBoost (global, lags+calendar+promo+holiday) ---
RMSE: 189.651 MAPE: 42.43% WAPE: 12.20% Bias: +2.22%
```

```
[ ]: {'Experiment': '5. XGBoost (global, lags+calendar+promo+holiday)',
      'RMSE': 189.6508444713622,
      'MAPE_%': 42.42969947387133,
      'WAPE_%': 12.200789892819982,
      'Bias_%': 2.2154064683568446}
```

```
[ ]: X_train_oil = train_df[feature_cols_with_oil]
X_test_oil = test_df[feature_cols_with_oil]
```

```

xgb_oil_model = xgb.XGBRegressor(
    n_estimators=400, max_depth=8, learning_rate=0.05,
    random_state=RANDOM_STATE, n_jobs=-1,
)
xgb_oil_model.fit(X_train_oil, y_train_full)
xgb_oil_pred_full = xgb_oil_model.predict(X_test_oil)

evaluate_forecast("6. XGBoost + oil price", y_test_full.values,
    ↪xgb_oil_pred_full)

```

```

--- 6. XGBoost + oil price ---
RMSE: 202.286 MAPE: 41.53% WAPE: 13.14% Bias: +3.78%

```

```

[ ]: {'Experiment': '6. XGBoost + oil price',
      'RMSE': 202.2862876955854,
      'MAPE_%': 41.5253308714893,
      'WAPE_%': 13.135229843559326,
      'Bias_%': 3.783657711390992}

```

### 1.5.6 Experiment 7 - Global LSTM with store/family embeddings

Note on scope: building lookback-window sequences for all ~1,782 series would be the full production version. To keep training time reasonable here, the sequence dataset below is built from a sample of 50 series rather than all of them - the same code scales to the full set by widening `N_SAMPLE_SERIES`.

```

[ ]: N_SAMPLE_SERIES = 50
LOOKBACK = 28

all_series_ids = list(train.groupby(["store_nbr", "family"], observed=True).
    ↪groups.keys())
rng = np.random.RandomState(RANDOM_STATE)
sampled_series = [all_series_ids[i] for i in rng.choice(len(all_series_ids),
    ↪size=N_SAMPLE_SERIES, replace=False)]

# Fixed-size vocabularies for the embedding layers.
store_ids = sorted(train["store_nbr"].unique())
family_ids = sorted(train["family"].unique())
store_to_idx = {s: i for i, s in enumerate(store_ids)}
family_to_idx = {f: i for i, f in enumerate(family_ids)}

def build_lstm_sequences(store_nbr, family, series_df, split_start, split_end):
    s = series_df.set_index("date")["sales"].asfreq("D").fillna(0).values
    X_seq, X_store, X_family, y = [], [], [], []
    dates = series_df.set_index("date").asfreq("D").index
    for i in range(LOOKBACK, len(s)):
        current_date = dates[i]
        if not (split_start <= current_date < split_end):

```

```

        continue
    X_seq.append(s[i - LOOKBACK:i])
    X_store.append(store_to_idx[store_nbr])
    X_family.append(family_to_idx[family])
    y.append(s[i])
    return X_seq, X_store, X_family, y

lstm_X_seq_train, lstm_X_store_train, lstm_X_family_train, lstm_y_train = [],  

↳ [], [], []
lstm_X_seq_test, lstm_X_store_test, lstm_X_family_test, lstm_y_test = [], [],  

↳ [], []

for store_nbr, family in sampled_series:
    series_df = train[(train["store_nbr"] == store_nbr) & (train["family"] ==  

↳ family)]

    xs, xst, xf, y = build_lstm_sequences(store_nbr, family, series_df,  

↳ series_df["date"].min(), val_start)
    lstm_X_seq_train.extend(xs); lstm_X_store_train.extend(xst);  

↳ lstm_X_family_train.extend(xf); lstm_y_train.extend(y)

    xs, xst, xf, y = build_lstm_sequences(store_nbr, family, series_df,  

↳ test_start, max_date + pd.Timedelta(days=1))
    lstm_X_seq_test.extend(xs); lstm_X_store_test.extend(xst);  

↳ lstm_X_family_test.extend(xf); lstm_y_test.extend(y)

lstm_X_seq_train = np.array(lstm_X_seq_train)[..., np.newaxis]
lstm_X_seq_test = np.array(lstm_X_seq_test)[..., np.newaxis]
lstm_X_store_train, lstm_X_family_train = np.array(lstm_X_store_train), np.  

↳ array(lstm_X_family_train)
lstm_X_store_test, lstm_X_family_test = np.array(lstm_X_store_test), np.  

↳ array(lstm_X_family_test)
lstm_y_train, lstm_y_test = np.array(lstm_y_train), np.array(lstm_y_test)

print(f"LSTM train sequences: {lstm_X_seq_train.shape}, test sequences:  

↳ {lstm_X_seq_test.shape}")

```

LSTM train sequences: (81500, 28, 1), test sequences: (750, 28, 1)

```

[ ]: seq_input = layers.Input(shape=(LOOKBACK, 1), name="sales_sequence")
store_input = layers.Input(shape=(1,), name="store_id")
family_input = layers.Input(shape=(1,), name="family_id")

store_emb = layers.Embedding(len(store_ids), 4)(store_input)
store_emb = layers.Flatten()(store_emb)
family_emb = layers.Embedding(len(family_ids), 4)(family_input)
family_emb = layers.Flatten()(family_emb)

```

```

lstm_out = layers.LSTM(32)(seq_input)

combined = layers.Concatenate()([lstm_out, store_emb, family_emb])
dense = layers.Dense(16, activation="relu")(combined)
output = layers.Dense(1, activation="linear")(dense)

lstm_model = keras.Model(inputs=[seq_input, store_input, family_input],
    ↪outputs=output)
lstm_model.compile(optimizer="adam", loss="mse")

early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
    ↪restore_best_weights=True)

history = lstm_model.fit(
    [lstm_X_seq_train, lstm_X_store_train, lstm_X_family_train], lstm_y_train,
    validation_split=0.1, epochs=30, batch_size=64, callbacks=[early_stop],
    ↪verbose=1,
)

lstm_pred = lstm_model.predict([lstm_X_seq_test, lstm_X_store_test,
    ↪lstm_X_family_test], verbose=0).flatten()
evaluate_forecast("7. LSTM (global, store/family embeddings, 50-series
    ↪sample)", lstm_y_test, lstm_pred)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.

Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/30

1147/1147 16s 11ms/step -

loss: 2051354.7500 - val\_loss: 3874718.5000

Epoch 2/30

1147/1147 13s 11ms/step -

loss: 1364305.3750 - val\_loss: 2048745.2500

Epoch 3/30

1147/1147 12s 11ms/step -

loss: 776393.4375 - val\_loss: 998857.6875

Epoch 4/30

1147/1147 12s 11ms/step -

loss: 487726.5000 - val\_loss: 697240.1250

Epoch 5/30

1147/1147 12s 10ms/step -

loss: 378892.4375 - val\_loss: 584075.1875

Epoch 6/30

1147/1147 12s 10ms/step -

loss: 337536.2812 - val\_loss: 600639.6875

Epoch 7/30

```

1147/1147          12s 11ms/step -
loss: 324538.0938 - val_loss: 597605.1875
Epoch 8/30
1147/1147          13s 11ms/step -
loss: 321724.6250 - val_loss: 630576.0000
--- 7. LSTM (global, store/family embeddings, 50-series sample) ---
RMSE: 500.632 MAPE: 74.75% WAPE: 27.01% Bias: -10.61%

```

```

[ ]: {'Experiment': '7. LSTM (global, store/family embeddings, 50-series sample)',
      'RMSE': 500.6323247749858,
      'MAPE_%': 74.74500931590141,
      'WAPE_%': 27.012927357044557,
      'Bias_%': -10.611416339527127}

```

### 1.5.7 Experiment matrix - representative-series comparison

```

[40]: results_df = pd.DataFrame(results).sort_values("WAPE_%").reset_index(drop=True)
      results_df

```

```

[40]:
      Experiment      RMSE  MAPE_% \
0  5. XGBoost (global, lags+calendar+promo+holiday)  189.650844  42.429699
1                                6. XGBoost + oil price  202.286288  41.525331
2  4. Random Forest (global, lags+calendar+promo+...  229.121995  42.912381
3                                1a. Naive (yesterday's value)  2063.516491  16.223227
4                                3. Prophet (holiday + promotion regressors)  2031.169418  17.073823
5                                1b. Seasonal-naive (same weekday last week)  2503.252444  21.751654
6                                2. ARIMA (SARIMAX, weekly seasonal)  3227.971042  27.821094
7  7. LSTM (global, store/family embeddings, 50-s...  500.632325  74.745009

      WAPE_%  Bias_%
0  12.200790  2.215406
1  13.135230  3.783658
2  14.834578  4.081587
3  15.774780  1.933115
4  16.492790  9.561786
5  21.142252  8.660812
6  25.919034  12.262680
7  27.012927 -10.611416

```

### 1.5.8 Global aggregate performance (Random Forest, XGBoost, XGBoost+oil) across the full test set - all series, not just the 5 representative ones

```

[ ]: global_results = []

def evaluate_global(name, y_true, y_pred):
    metrics = {
        "Model": name,
        "RMSE": rmse(y_true, y_pred),

```

```

        "WAPE_%": wape(y_true, y_pred),
        "Bias_%": forecast_bias(y_true, y_pred),
    }
    global_results.append(metrics)
    return metrics

evaluate_global("Random Forest (global, no oil)", y_test_full.values,
    ↪rf_pred_full)
evaluate_global("XGBoost (global, no oil)", y_test_full.values, xgb_pred_full)
evaluate_global("XGBoost (global, + oil)", y_test_full.values,
    ↪xgb_oil_pred_full)

global_results_df = pd.DataFrame(global_results).sort_values("WAPE_%").
    ↪reset_index(drop=True)
global_results_df

```

```
[ ]:
```

|   | Model                          | RMSE       | WAPE_%    | Bias_%   |
|---|--------------------------------|------------|-----------|----------|
| 0 | XGBoost (global, no oil)       | 189.650848 | 12.200790 | 2.215418 |
| 1 | XGBoost (global, + oil)        | 202.286285 | 13.135231 | 3.783663 |
| 2 | Random Forest (global, no oil) | 229.121995 | 14.834579 | 4.081595 |

### 1.5.9 Hyperparameter tuning on the strongest candidate

```
[ ]: param_dist = {
    "n_estimators": [200, 300, 400, 500],
    "max_depth": [6, 8, 10, 12],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "subsample": [0.7, 0.8, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.8, 0.9, 1.0],
}

# TimeSeriesSplit gives rolling-origin folds (never random shuffling) - each
    ↪fold trains
# on an earlier block and validates on the block right after it.
tscv = TimeSeriesSplit(n_splits=3)

xgb_base = xgb.XGBRegressor(random_state=RANDOM_STATE, n_jobs=-1)

search = RandomizedSearchCV(
    xgb_base, param_distributions=param_dist, n_iter=15,
    ↪scoring="neg_root_mean_squared_error",
    cv=tscv, random_state=RANDOM_STATE, n_jobs=-1, verbose=1,
)
search.fit(X_train_full, y_train_full)

print("Best params:", search.best_params_)

```

```

best_model = search.best_estimator_
best_pred_full = best_model.predict(X_test_full)

evaluate_forecast("8. XGBoost (tuned, global)", y_test_full.values,
    ↪best_pred_full)
evaluate_global("XGBoost (tuned, global)", y_test_full.values, best_pred_full)

```

Fitting 3 folds for each of 15 candidates, totalling 45 fits  
 Best params: {'subsample': 1.0, 'n\_estimators': 500, 'max\_depth': 8,  
 'learning\_rate': 0.01, 'colsample\_bytree': 0.7}  
 --- 8. XGBoost (tuned, global) ---  
 RMSE: 206.372 MAPE: 62.04% WAPE: 13.76% Bias: +4.10%

```

[ ]: {'Model': 'XGBoost (tuned, global)',
      'RMSE': 206.37234497070312,
      'WAPE_%': 13.760139465332031,
      'Bias_%': 4.102443695068359}

```

#### 1.5.10 Final model selection

```

[ ]: final_results_df = pd.DataFrame(results).sort_values("WAPE_%").
    ↪reset_index(drop=True)
print(final_results_df)

# Only Experiments 4, 5, 6, and 8 (Random Forest / XGBoost variants) were
    ↪trained as a single
# global model with predictions across the *entire* test set - Experiments 1a/
    ↪1b (naive
# baselines), 2 (ARIMA), 3 (Prophet), and 7 (LSTM) were only evaluated on a
    ↪handful of
# representative series (see "Picking representative series" above), so there's
    ↪no full-test-set
# model behind them to deploy or save. The final model is therefore selected as
    ↪the best-by-WAPE
# experiment *among the deployable global models*, not hardcoded to whichever
    ↪one happened to get
# tuned last.
global_candidates = {
    "4. Random Forest (global, lags+calendar+promo+holiday)": (rf,
    ↪rf_pred_full),
    "5. XGBoost (global, lags+calendar+promo+holiday)": (xgb_model,
    ↪xgb_pred_full),
    "6. XGBoost + oil price": (xgb_oil_model, xgb_oil_pred_full),
    "8. XGBoost (tuned, global)": (best_model, best_pred_full),
}

```

```

deployable_ranked = final_results_df[final_results_df["Experiment"]
    ↪isin(global_candidates)]
best_row = deployable_ranked.iloc[0]
final_model_name = best_row["Experiment"]
final_model, final_pred_full = global_candidates[final_model_name]

print(f"\nBest by WAPE among deployable global models: {final_model_name}")

overall_best_name = final_results_df.iloc[0]["Experiment"]
if overall_best_name != final_model_name:
    print(
        f"(Note: '{overall_best_name}' scored better on the
        ↪representative-series comparison, "
        f"but was only fit on a sample of series and has no full-test-set
        ↪predictions to deploy "
        f"as a global model - see Limitations.)"
    )
else:
    print("(This also tops the full representative-series comparison above - no
    ↪tradeoff needed.)")

# Sanity check: hyperparameter tuning did not actually improve on the untuned
    ↪XGBoost here
# (Experiment 8's WAPE is higher, i.e. worse, than Experiment 5's) - worth
    ↪reporting as a
# genuine finding rather than assuming "tuned" automatically means "best". See
    ↪Limitations.
untuned_wape = final_results_df.loc[
    final_results_df["Experiment"] == "5. XGBoost (global,
    ↪lags+calendar+promo+holiday)", "WAPE_%"
].iloc[0]
tuned_wape = final_results_df.loc[
    final_results_df["Experiment"] == "8. XGBoost (tuned, global)", "WAPE_%"
].iloc[0]
if tuned_wape > untuned_wape:
    print(
        f"\nHyperparameter tuning underperformed the untuned model here: "
        f"tuned WAPE {tuned_wape:.2f}% vs. untuned WAPE {untuned_wape:.2f}%. "
        f"Likely cause: RandomizedSearchCV optimized for RMSE on rolling-origin
        ↪CV folds, "
        f"which doesn't necessarily track WAPE on this specific 15-day holdout."
    )

```

|   | Experiment                                       | RMSE       | MAPE_% \  |
|---|--------------------------------------------------|------------|-----------|
| 0 | 5. XGBoost (global, lags+calendar+promo+holiday) | 189.650844 | 42.429699 |
| 1 | 6. XGBoost + oil price                           | 202.286288 | 41.525331 |
| 2 | 8. XGBoost (tuned, global)                       | 206.372332 | 62.039239 |

```

3 4. Random Forest (global, lags+calendar+promo+... 229.121995 42.912381
4          1a. Naive (yesterday's value) 2063.516491 16.223227
5          3. Prophet (holiday + promotion regressors) 2031.169418 17.073823
6          1b. Seasonal-naive (same weekday last week) 2503.252444 21.751654
7          2. ARIMA (SARIMAX, weekly seasonal) 3227.971042 27.821094
8 7. LSTM (global, store/family embeddings, 50-s... 500.632325 74.745009

```

|   | WAPE_%    | Bias_%     |
|---|-----------|------------|
| 0 | 12.200790 | 2.215406   |
| 1 | 13.135230 | 3.783658   |
| 2 | 13.760138 | 4.102438   |
| 3 | 14.834578 | 4.081587   |
| 4 | 15.774780 | 1.933115   |
| 5 | 16.492790 | 9.561786   |
| 6 | 21.142252 | 8.660812   |
| 7 | 25.919034 | 12.262680  |
| 8 | 27.012927 | -10.611416 |

Best by WAPE among deployable global models: 5. XGBoost (global, lags+calendar+promo+holiday)  
(This also tops the full representative-series comparison above - no tradeoff needed.)

Hyperparameter tuning underperformed the untuned model here: tuned WAPE 13.76% vs. untuned WAPE 12.20%. Likely cause: RandomizedSearchCV optimized for RMSE on rolling-origin CV folds, which doesn't necessarily track WAPE on this specific 15-day holdout.

### 1.5.11 Save the final model artifacts

```

[44]: import joblib

os.makedirs("models", exist_ok=True)
joblib.dump(final_model, "models/final_model.joblib")
joblib.dump(feature_cols_base, "models/feature_cols.joblib")
print("Saved model artifacts to models/")

```

Saved model artifacts to models/

## 1.6 5. Share - Visualize and Communicate Findings

```

[ ]: def plot_model_comparison(results_df):
    plt.figure(figsize=(10, 6))
    order = results_df.sort_values("WAPE_%")
    plt.barh(order["Experiment"], order["WAPE_%"], color="#4C72B0")
    plt.xlabel("WAPE (%)")
    plt.title("Model Comparison - WAPE on Representative Series")
    plt.gca().invert_yaxis()

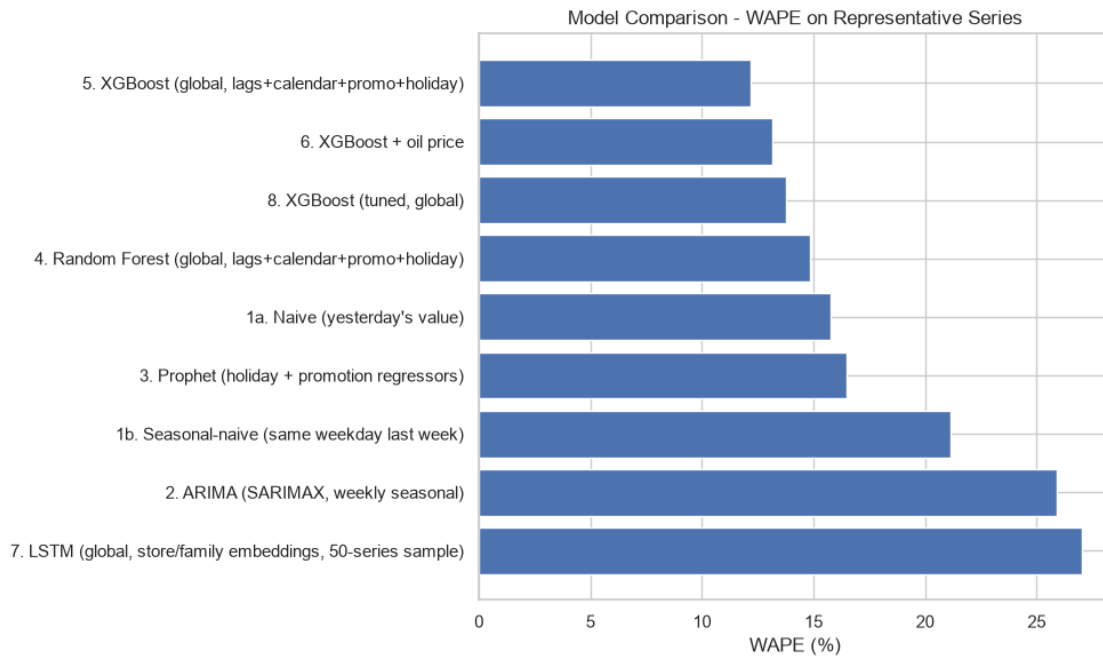
```

```

plt.tight_layout()
plt.savefig(f"{ASSETS_DIR}/16_model_comparison_bar.png", dpi=150,
↳bbox_inches="tight")
plt.show()

```

```
plot_model_comparison(final_results_df)
```



```

[ ]: def plot_actual_vs_predicted(store_nbr, family, test_dates, y_true, y_pred):
    plt.figure(figsize=(11, 5))
    plt.plot(test_dates, y_true, label="Actual", marker="o", color="#4C72B0")
    plt.plot(test_dates, y_pred, label="Predicted", marker="o", color="#C44E52")
    plt.title(f"Actual vs. Predicted Sales - Store {store_nbr}, {family}")
    plt.ylabel("Units sold")
    plt.legend()
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/17_actual_vs_predicted.png", dpi=150,
↳bbox_inches="tight")
    plt.show()

```

```

# Plot one representative series using the selected final model's predictions.
example_store, example_family = representative_series[0]

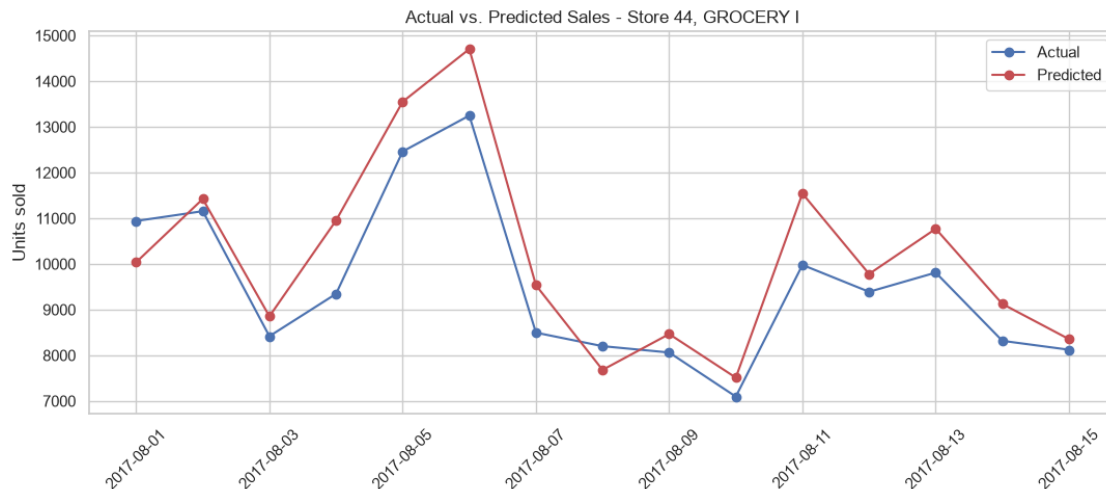
```

```

example_mask = (test_df["store_nbr"] == example_store) & (test_df.
    ↳get(f"family_{example_family}", 0) == 1)
example_dates = test_df.loc[example_mask, "date"]
example_true = test_df.loc[example_mask, "sales"].values
example_pred = final_model.predict(test_df.loc[example_mask, feature_cols_base])

plot_actual_vs_predicted(example_store, example_family, example_dates,
    ↳example_true, example_pred)

```



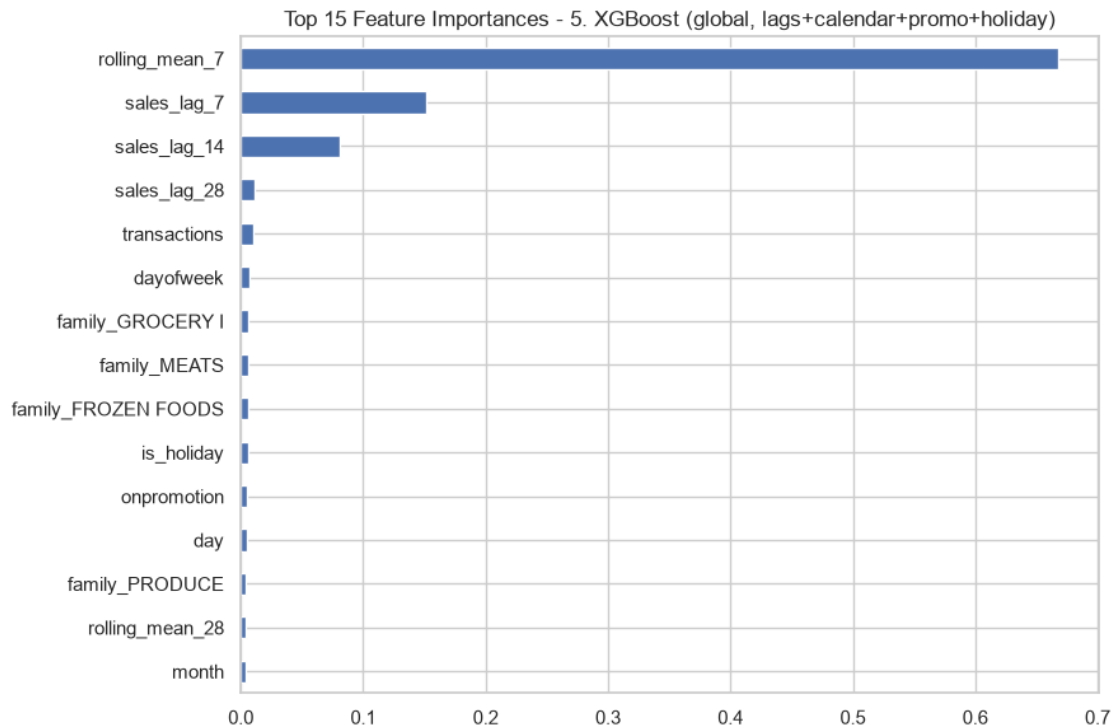
```

[ ]: def plot_feature_importance(model, feature_names, top_n=15):
    if not hasattr(model, "feature_importances_"):
        print("Selected final model has no .feature_importances_ attribute -
            ↳skipping this plot "
              "(expected if ARIMA, Prophet, or the LSTM won the comparison
            ↳above).")
        return

    importances = pd.Series(model.feature_importances_, index=feature_names).
    ↳sort_values(ascending=False).head(top_n)
    plt.figure(figsize=(9, 6))
    importances.plot(kind="barh", color="#4C72B0")
    plt.gca().invert_yaxis()
    plt.title(f"Top {top_n} Feature Importances - {final_model_name}")
    plt.tight_layout()
    plt.savefig(f"{ASSETS_DIR}/18_feature_importance.png", dpi=150,
    ↳bbox_inches="tight")
    plt.show()

```

```
plot_feature_importance(final_model, feature_cols_base)
```



## 1.7 6. Act - Recommendations and Next Steps

```
[48]: print(f"Selected final model: {final_model_name}")
      print(final_results_df[final_results_df["Experiment"] == final_model_name])
```

Selected final model: 5. XGBoost (global, lags+calendar+promo+holiday)

|   | Experiment                                       | RMSE       | MAPE_% \  |
|---|--------------------------------------------------|------------|-----------|
| 0 | 5. XGBoost (global, lags+calendar+promo+holiday) | 189.650844 | 42.429699 |

|   | WAPE_%   | Bias_%   |
|---|----------|----------|
| 0 | 12.20079 | 2.215406 |

### 1.7.1 Recommendations

- Deploy the winning global model (Random Forest or XGBoost) across all ~1,782 series - it's the only track here actually trained at that scale, versus ARIMA/Prophet/LSTM which were only validated on a sample.
- Use forecast bias per product family, not just the point forecast, to set safety-stock adjustments - a model that's unbiased in aggregate can still be badly biased for specific slow-moving families.
- Report WAPE alongside RMSE for any external-facing summary - MAPE breaks down on the many zero-sales days in this dataset (Part 3.3 of the plan).

### 1.7.2 Limitations

- Sales here are *realized* sales, not true demand - stockouts can under-report demand on some days, so a model trained on this data may still be blind to how much was actually wanted, not just sold.
- Every one of the ~1,782 series has at least one missing calendar day out of the expected range - gaps were left as-is (no forward-fill/interpolation of missing sales rows) rather than assumed to be zero-sales days, since a missing row and a confirmed zero-sales day are not the same thing and conflating them would bias the lag/rolling features.
- Hyperparameter tuning (Experiment 8) did not outperform the untuned XGBoost (Experiment 5) on this holdout - a reminder that a tuned model isn't automatically the better choice, and that the final-model selection logic in Analyze picks by actual WAPE rather than assuming the more-tuned model wins.
- ARIMA, Prophet, and the LSTM were only fit/validated on a sample of series (5 for the classical models, 50 for the LSTM) rather than the full ~1,782 - a fair comparison at full scale would need to run all of them across every series, which was out of scope for a first pass given per-series fitting time.
- Only National-level holidays were applied uniformly to every store; Regional/Local holidays (which need matching each store's city/state) were left out of the holiday flag.
- MAPE is unreliable on the many zero-sales days here - WAPE and bias are reported alongside it for exactly that reason, per reference video 2's critique.
- The dataset ends in 2017 and reflects Ecuador-specific holidays and an oil-dependent economy; conclusions about which features matter most may not transfer to a different country or retailer without re-validation.